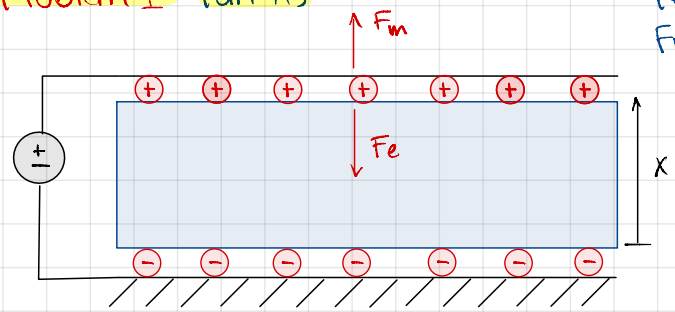
Problem 1 Part A)



Fe - caused by charges on electrodes
Fm - mech force caused by elasticity of dielectric

Corrections made in this color!!

Capacitance: $C = \frac{\varepsilon_0 \varepsilon_r A}{x}$

Total Energy of System $\Pi = -\Pi_e + \Pi_m = \frac{1}{2}CV^2 + \frac{1}{2}k(x_0-x)^2 = \frac{1}{2}\left(\frac{\varepsilon_0\varepsilon_r A}{x}\right)V^2 + \frac{1}{2}k(x_0-x)^2$

$\frac{d\Pi}{dx} = 0 \Rightarrow \frac{d\Pi_e}{dx} = -\frac{1}{2}\frac{\varepsilon_0\varepsilon_r A}{x^2}V^2 = F_e$ & $\frac{d\Pi_m}{dx} = -k(x_0-x) = F_m$

$F_m = F_e \Rightarrow \frac{1}{2}\frac{\varepsilon_0\varepsilon_r A}{x^2}V^2 = k(x_0-x)$ (1)

$\frac{dF_m}{dx} = \frac{dF_e}{dx} \Rightarrow -\frac{\varepsilon_0\varepsilon_r A V^2}{x^3} = -k \Rightarrow K = \frac{\varepsilon_0\varepsilon_r A V^2}{x^3}$ @ pull-in $k = \frac{\varepsilon_0\varepsilon_r A V_p^2}{x_p^3}$ (2)

(1) & (2) $\Rightarrow k(x_0-x_p) = \frac{1}{2}\frac{\varepsilon_0\varepsilon_r A V_p^2}{x_p^2} \Rightarrow k(x_0-x_p) = \frac{1}{2}kx_p \Rightarrow x_0 - x_p = \frac{1}{2}x_p \Rightarrow x_0 = \frac{3}{2}x_p \therefore x_p = \frac{2}{3}x_0$

(2) $\Rightarrow k = \frac{\varepsilon_0\varepsilon_r A V_p^2}{x_p^3} \Rightarrow \frac{kx_p^3}{\varepsilon_0\varepsilon_r A} = V_p^2 \Rightarrow V_p = \sqrt{\frac{k(\frac{2}{3}x_0)^3}{\varepsilon_0\varepsilon_r A}}$ ✓ $V_p = \sqrt{\frac{8kx_0^3}{27\varepsilon_0\varepsilon_r A}}$

Part C Pull-in happens when the electrostatic attractive force exceeds the mechanical restoring force. At that point the actuator collapses and the system fails. In this case:

$d_{max} = x_0 - x_p = x_0 - \frac{2}{3}x_0 = \frac{1}{3}x_0 = \frac{1}{3}(220\mu m) = 73.3\mu m$

$F_m = k(x_0-x) = F_e$ @ $V = V_p$

$x_p = 1.47\times10^{-4} m$

Problem 2 Part A

$\lambda_1 = \frac{x}{x_0}$, $\lambda_2 = \lambda_3 = \lambda$, $\lambda_1\cdot\lambda_2\cdot\lambda_3 = 1$, $\lambda_2 = \lambda_3 = \frac{1}{\sqrt{\lambda_1}} = \frac{1}{\sqrt{x/x_0}} = \sqrt{\frac{x_0}{x}}$

$W = C_1(\lambda_1^2+\lambda_2^2+\lambda_3^2-3) = C_1\left(\left(\frac{x}{x_0}\right)^2 + 2\left(\sqrt{\frac{x_0}{x}}\right)^2 - 3\right) = C_1\left(\left(\frac{x}{x_0}\right)^2 + 2\left(\frac{x_0}{x}\right) - 3\right)$

$A = \frac{A_0}{\lambda}$, $F_e = \frac{\varepsilon_0\varepsilon_r A V^2}{2x_0^2\lambda^3}$

$\Pi_m = WV_0 = WA_0x_0 = C_1\left(\left(\frac{x}{x_0}\right)^2 + 2\left(\frac{x_0}{x}\right) - 3\right)A_0x_0$

$= \frac{1}{3}A_0E\left(\lambda_1 - \frac{1}{\lambda^2}\right)$

$F_m = \frac{d\Pi_m}{dx} = \underset{E/6}{C_1}\left(2\left(\frac{x}{x_0^2}\right) - 2\frac{x_0}{x^2} - 0\right)A_0x_0 \Rightarrow F_m = 2C_1A_0\left(\frac{x}{x_0} - \frac{x_0^2}{x^2}\right) \Rightarrow F_m = \frac{1}{3}A_0\left(\lambda_1 - \frac{1}{\lambda_1^2}\right)$

Part C At small displacements the Neo-Hookean model predicts higher mechanical resistance than the linear model. This is due to material stiffness, which limits how far the actuator can compress before the restoring force balances the electrostatic force.

Problem 3

$\rightarrow F_{en} = nF_e = n\frac{1}{2}\frac{\varepsilon_0\varepsilon_r A_t V^2}{x^2}$

Part A $t = nx$, $x = t_0$, $x_{LR} = t_0L_0R_0$, Assume incompressible $\rightarrow V_0 = V = 2\pi t_0L_0R_0$

$\therefore x = \frac{2\pi t_0L_0R_0}{2\pi LR} = \frac{t_0L_0R_0}{LR}$ $\lambda_1\cdot\lambda_2\cdot\lambda_3 = 1$ $\lambda_1 = \frac{x}{x_0} = \frac{x}{t_0}$ $\lambda_2 = \lambda_3 = \lambda = \sqrt{\frac{1}{\lambda_1}} = \sqrt{\frac{1}{x/t_0}}$

Neo-Hookean $\rightarrow W = C_1(\lambda_1^2+\lambda_2^2+\lambda_3^2-3)$ $C_1 = \frac{E}{6}$, $V_0 = 2\pi t_0L_0R_0$

$\Rightarrow \Pi_m = WV_0$

$F_m = \frac{d\Pi_m}{dx}$

$$\Rightarrow F_m = F_e = \frac{n\varepsilon_0\varepsilon_r A_t V^2}{x^2} \qquad \Rightarrow W = C_1\left(\left(\frac{x}{t_0}\right)^2 + \left(\sqrt{\frac{t_0}{x}}\right)^2 + \left(\sqrt{\frac{t_0}{x}}\right)^2 - 3\right)$$

$$= C_1\left(\frac{x^2}{t_0^2} + 2\left(\frac{t_0}{x}\right) - 3\right)$$

$$\Rightarrow \Pi_m = WV_0 = C_1V_0\left(\frac{x^2}{t_0^2} + 2\left(\frac{t_0}{x}\right) - 3\right)$$

$$\Rightarrow \frac{d\Pi_m}{dx} = F_m = C_1V_0\left(\frac{2x}{t_0^2} - 2\frac{t_0}{x^2}\right) = 2C_1V_0\left(\frac{x}{t_0^2} - \frac{t_0}{x^2}\right)$$

$$F_e = \frac{n(2\pi R_0L)\varepsilon_0\varepsilon_r V^2}{2x^2} = \frac{n\pi R_0L\varepsilon_0\varepsilon_r V^2}{x^2} \qquad X = \frac{t_0L_0R_0}{LR}$$

$$F_m = 2(2\pi t_0L_0R_0)C_1\left(\frac{t_0L_0R_0}{LRt_0^2} - \frac{t_0L^2R^2}{t_0^2L_0^2R_0^2}\right), \text{ \& } F_e = \frac{n\pi R_0L\varepsilon_0\varepsilon_r V^2L^2R^2}{t_0^2L_0^2R_0^2}$$

$$\Rightarrow 4\pi t_0L_0R_0C_1\left(\frac{L_0R_0}{LRt_0} - \frac{L^2R^2}{t_0L_0^2R_0^2}\right) = \frac{n\pi L^3R^2\varepsilon_0\varepsilon_r V^2}{t_0^2L_0^2R_0}$$

$$\Rightarrow V = \sqrt{\frac{4\pi t_0L_0R_0C_1}{n\pi L^3R^2\varepsilon_0\varepsilon_r}\left(\frac{L_0R_0}{LRt_0} - \frac{L^2R^2}{t_0L_0^2R^2}\right)\cdot t_0^2L_0^2R_0} \Rightarrow \boxed{V = \sqrt{\frac{4t_0^3L_0^3R_0^2C_1}{nL^3R^2\varepsilon_0\varepsilon_r}\left(\frac{L_0R_0}{LRt_0} - \frac{L^2R^2}{t_0L_0^2R^2}\right)}}$$

$|F_m| = |F_e|$

$$\frac{E}{3}A_0\left|\lambda - \frac{1}{\lambda^2}\right| = \frac{\varepsilon_0\varepsilon_r AU^2}{2x^2}$$

$$\frac{E}{3}A_0\left|\lambda - \frac{1}{\lambda^2}\right| = \frac{\varepsilon_0\varepsilon_r A_0U^2}{2x^2\lambda^3}$$

$A = \frac{A_0}{\lambda}$

$$\Rightarrow U^2 = \frac{2Ex_0^2\lambda^3}{3\varepsilon_0\varepsilon_r}\left|\lambda - \frac{1}{\lambda^2}\right|$$

$$U = \sqrt{\frac{2Ex_0^2\lambda^3}{3\varepsilon_0\varepsilon_r}\left|1 - \frac{1}{\lambda^2}\right|}$$

Part C Static vs Dynamic response. Our model is a static equilibrium (no resonance amplification) so it will generally predict higher voltages for the same displacement. We got ~~2.3kV~~ compared to ~~1.3kV~~ on the research paper. We used a single layer thickness to which does not reflect the real model from the research paper. Our model is off by a scale of around 1.5 to 1.7 from the research data

Part D The steady-state pull-in voltage corresponds to the maximum of the voltage-length curve. After this point the actuator becomes unstable and snaps to a fully actuated state, so the predicted pull-in voltage is the maximum value of V(L) computed from the model

"pull-in" voltage U_p, $|F_m(\lambda_p)| = |F_e(\lambda_p)|$ and $\frac{d|F_m|}{d\lambda} = \frac{d|F_e|}{d\lambda}$ @ $\lambda = \lambda_p$

$$\Rightarrow |F_m| = |F_e| \Rightarrow \frac{E}{3}\left(\frac{1}{\lambda_p^2} - \lambda_p\right) = \frac{\varepsilon_0\varepsilon_r U^2}{2x_0^2\lambda_p^3} \quad \text{take derivative w.r.t } \lambda$$

$$\left.\frac{d|F_m|}{d\lambda}\right|_{\lambda=\lambda_p} = \frac{E}{3}A_0\left(\frac{-2}{\lambda_p^3} - 1\right)$$

$$\left.\frac{d|F_e|}{d\lambda}\right|_{\lambda=\lambda_p} = -\frac{3\varepsilon_0\varepsilon_r A_0U^2}{2x_0^2\lambda_p^4}$$

$$\frac{E}{3}\left(\frac{-2}{\lambda_p^3} - 1\right) = \frac{-3\varepsilon_0\varepsilon_r U^2}{2x_0^2\lambda_p^4}$$

$$\boxed{\lambda_p = 0.629 \text{ \& } U = 1325.505\text{ V}}$$

Open in Colab

Problem 1: Modeling Dielectric Elastomer Actuators - Linear Elasticity (30 points)

In class we developed a model of DEAs assuming linear elasticity. We generated models for the elastic and electrical forces acting on a DEA, and considered when these forces are balanced, for various applied voltages.

a) We ended by saying that a "pull-in" voltage, V_p, the $|F|$ vs. x plots intersect at a single point. According to this model, what is the pull-in voltage for a DEA, and what is the corresponding position, x_p? Write your answers in terms of $k, x_0, \epsilon_0, \epsilon_r$ and A as defined in lecture.

b) Given the following parameter values, plot $|F_m|$ and $|F_e|$ vs. x for two voltages: V_p and some lower voltage $0 < V < V_p$:

- modulus $E = 140$ kPa
- original thickness $x_0 = 220$ μm
- permittivity of free space $\epsilon_0 = 8.854 \times 10^{-12}$ F/m
- relative permittivity of free space $\epsilon_r = 2.8$
- actuator width $y_0 = 8$ mm
- actuator length $z_0 = 5$ cm

Recall: from linear elasticity that for Hooke's law, the slope is given by $k = EA/x_0$.

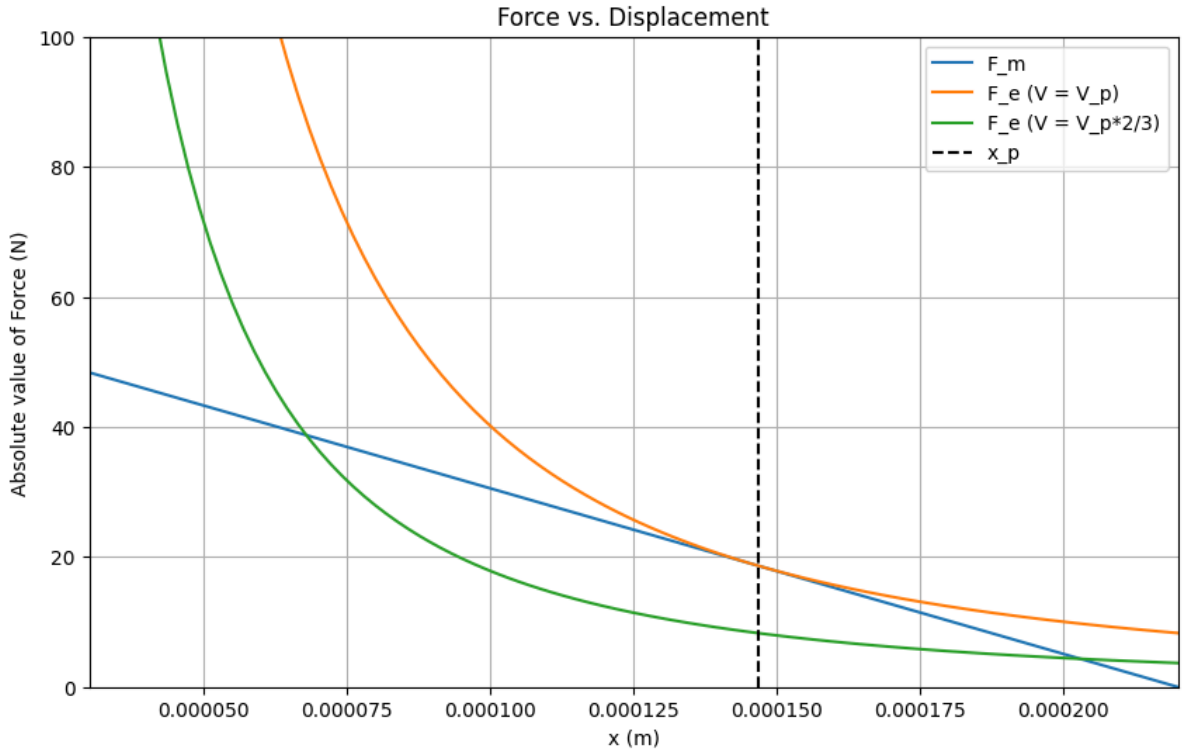
c) What is the maximum possible displacement of this DEA (just before failure)?

Problem 1 - Part B

In [2]:

```
import numpy as np
import matplotlib.pyplot as plt

# Problem 1 b)
E = 140e3  # Young's Modulus, Pa
x_0 = 220e-6  # Original thickness, m
eps_0 = 8.854e-12  # Permissivity of free space, Farad/m
eps_r = 2.8  # Relative permissivity, dimensionless
y_0 = 8e-3  # Actuator width, m
z_0 = 5e-2  # Actuator length, m

A = y_0 * z_0  # Actuator area, m^2 (assumed constant for this problem (Hookean
k = E * A / x_0  # Stiffness of actuator, N/m

# For sake of plot visibility, plot over x = (3e-5, x_0)
n = 100
x_LL = 3e-5
x = np.linspace(x_LL, x_0, n)

V_p = np.sqrt((8 * k * x_0**3) / (27 * eps_0 * eps_r * A))  # Pulldown voltage,
V_2 = V_p * 2/3  # Some voltage between V = 0 and V = V_p
x_p = x_0 * 2/3  # x where two curves intersect at V = V_p

# Initialize arrays
F_m = np.zeros(n)
F_e_V_p = np.zeros(n)
F_e_V_2 = np.zeros(n)

# Calculate forces
```

```
for i in range(n):
    F_m[i] = k * (x_0 - x[i])
    F_e_V_p[i] = (eps_0 * eps_r * A * V_p**2) / (2 * x[i]**2)
    F_e_V_2[i] = (eps_0 * eps_r * A * V_2**2) / (2 * x[i]**2)

# Create plot
plt.figure(figsize=(10, 6))
plt.plot(x, F_m, label='F_m')
plt.plot(x, F_e_V_p, label='F_e (V = V_p)')
plt.plot(x, F_e_V_2, label='F_e (V = V_p*2/3)')
plt.plot([x_p, x_p], [0, 100], 'k--', label='x_p')
plt.ylim([0, 100])
plt.xlim([x_LL, x_0])
plt.xlabel('x (m)')
plt.ylabel('Absolute value of Force (N)')
plt.legend()
plt.title('Force vs. Displacement')
plt.grid(True)
plt.show()

# Print some values for verification
print(f"Pulldown voltage V_p = {V_p:.2f} V")
print(f"Stiffness k = {k:.2f} N/m")
print(f"Intersection point x_p = {x_p:.2e} m")
```




```
Pulldown voltage V_p = 8999.14 V
Stiffness k = 254545.45 N/m
Intersection point x_p = 1.47e-04 m
```

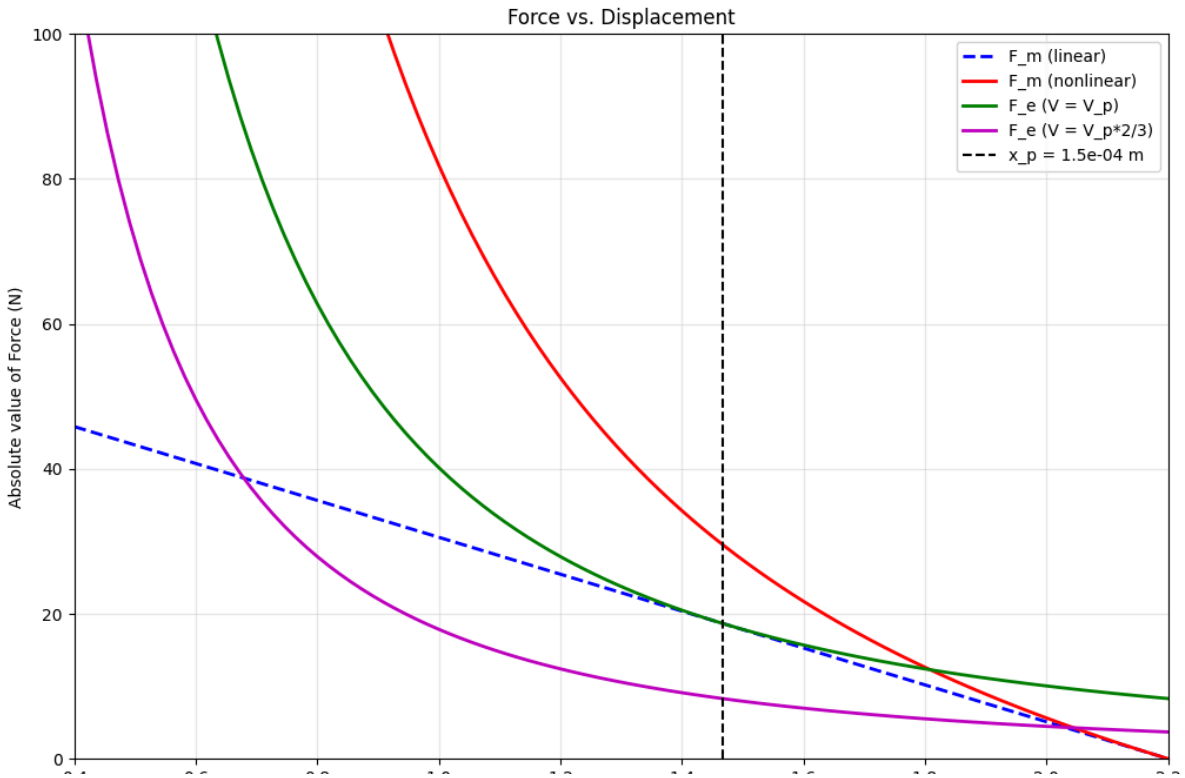
Problem 2 - Part B

In [1]:

```
import numpy as np
```

```
import matplotlib.pyplot as plt

# Problem 1 b)
E = 140e3  # Young's Modulus, Pa
x_0 = 220e-6  # Original thickness, m (220 µm)
eps_0 = 8.854e-12  # Permissivity of free space, Farad/m
eps_r = 2.8  # Relative permissivity, dimensionless
y_0 = 8e-3  # Actuator width, m
z_0 = 5e-2  # Actuator length, m

A_0 = y_0 * z_0  # Original actuator area, m^2
k = E * A_0 / x_0  # Stiffness of actuator, N/m (linear model)

# Use the exact x-range from your plot: 0.4e-4 to 2.2e-4 m
n = 100
x_LL = 0.4e-4  # 40 µm
x_UL = 2.2e-4  # 220 µm
x = np.linspace(x_LL, x_UL, n)

V_p = np.sqrt((8 * k * x_0**3) / (27 * eps_0 * eps_r * A_0))  # Pulldown voltag
V_2 = V_p * 2/3  # Some voltage between V = 0 and V = V_p
x_p = x_0 * 2/3  # x where two curves intersect at V = V_p

# Calculate lambda = x/x_0 (stretch ratio in thickness direction)
lambda_vals = x / x_0

# Calculate areas - for incompressible material, area scales with 1/lambda
A_lambda = A_0 / lambda_vals  # A = A_0/lambda

# Calculate forces - use ABSOLUTE VALUES as in the MATLAB plot
# Linear mechanical force (Hookean) - absolute value
F_m_linear = np.abs(k * (x_0 - x))

# Nonlinear mechanical force: Fm = (A * E / 3) * (lambda - 1/lambda^2)
# Take absolute value and ensure positive force
F_m_nonlinear = np.abs((A_0 * E / 3) * (lambda_vals - 1/lambda_vals**2))

# Electrical forces (using original area A_0 as in linear case) - already posit
F_e_V_p = (eps_0 * eps_r * A_0 * V_p**2) / (2 * x**2)
F_e_V_2 = (eps_0 * eps_r * A_0 * V_2**2) / (2 * x**2)

# Create plot
plt.figure(figsize=(12, 8))
plt.plot(x, F_m_linear, 'b--', label='F_m (linear)', linewidth=2)
plt.plot(x, F_m_nonlinear, 'r-', label='F_m (nonlinear)', linewidth=2)
plt.plot(x, F_e_V_p, 'g-', label='F_e (V = V_p)', linewidth=2)
plt.plot(x, F_e_V_2, 'm-', label='F_e (V = V_p*2/3)', linewidth=2)
plt.axvline(x=x_p, color='k', linestyle='--', label=f'x_p = {x_p:.1e} m')

plt.ylim([0, 100])
plt.xlim([x_LL, x_UL])
plt.xlabel('x (m)')
plt.ylabel('Absolute value of Force (N)')
plt.legend()
plt.title('Force vs. Displacement')
plt.grid(True, alpha=0.3)
```

```
# Add x10^-4 notation to x-axis as in your plot
from matplotlib.ticker import FuncFormatter
plt.gca().xaxis.set_major_formatter(FuncFormatter(lambda x, p: f'{x*1e4:.1f}'))
plt.gca().set_xlabel('x (m) $times 10^{-4}$')

plt.show()

# Print values for comparison and debugging
print(f"Pulldown voltage V_p = {V_p:.2f} V")
print(f"Stiffness k = {k:.2f} N/m")
print(f"Intersection point x_p = {x_p:.2e} m")
print(f"Original area A_0 = {A_0:.2e} m²")
print(f"Lambda range: {lambda_vals[0]:.3f} to {lambda_vals[-1]:.3f}")
print(f"F_m_linear range: {F_m_linear[0]:.2f} to {F_m_linear[-1]:.2f} N")
print(f"F_m_nonlinear range: {F_m_nonlinear[0]:.2f} to {F_m_nonlinear[-1]:.2f} I
print(f"F_e_V_p at x_min: {F_e_V_p[0]:.2f} N, at x_max: {F_e_V_p[-1]:.2f} N")
```




```
Lambda range: 0.182 to 1.000
F_m_linear range: 45.82 to 0.00 N
F_m_nonlinear range: 561.27 to 0.00 N
F_e_V_p at x_min: 250.96 N, at x_max: 8.30 N
```

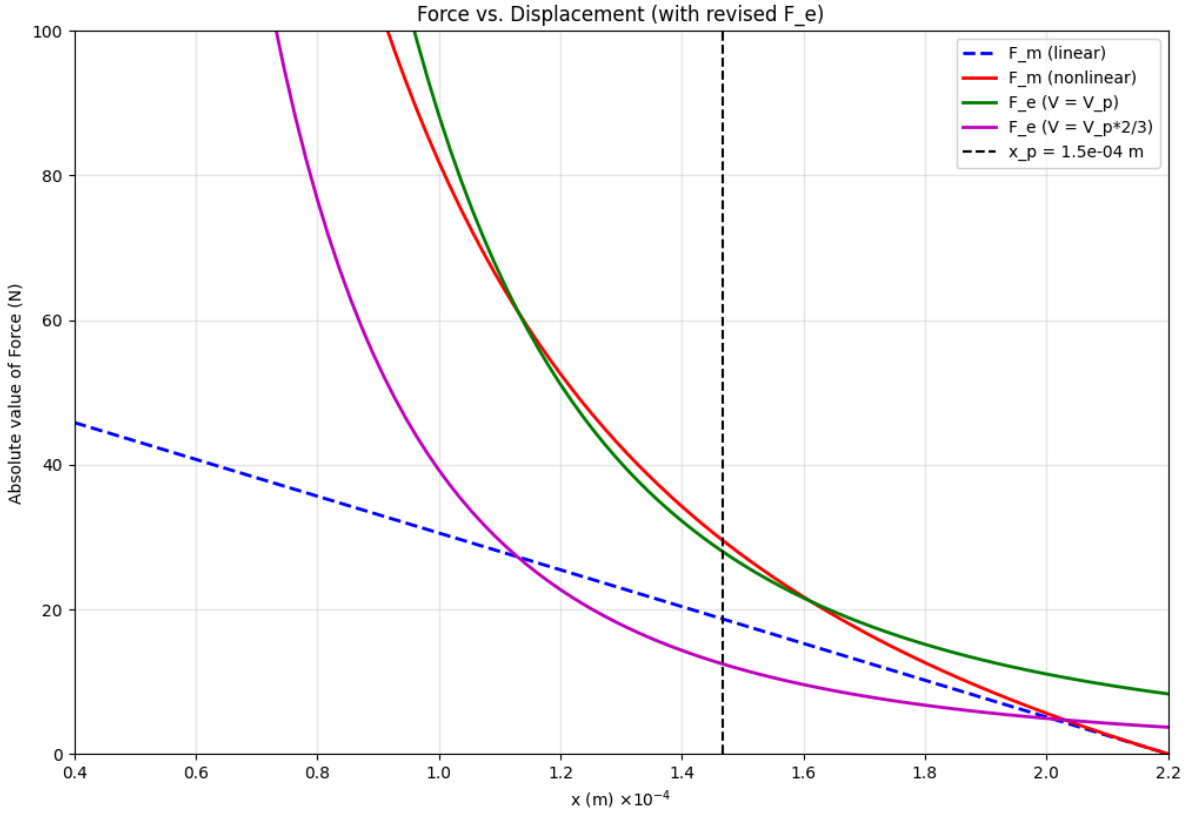
Problem 2 - Part C

In [2]:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Problem 1 b)
E = 140e3  # Young's Modulus, Pa
x_0 = 220e-6  # Original thickness, m (220 μm)
eps_0 = 8.854e-12  # Permissivity of free space, Farad/m
eps_r = 2.8  # Relative permissivity, dimensionless
y_0 = 8e-3  # Actuator width, m
z_0 = 5e-2  # Actuator length, m

A_0 = y_0 * z_0  # Original actuator area, m^2
k = E * A_0 / x_0  # Stiffness of actuator, N/m (linear model)

# Use the exact x-range from your plot: 0.4e-4 to 2.2e-4 m
n = 100
x_LL = 0.4e-4  # 40 μm
x_UL = 2.2e-4  # 220 μm
x = np.linspace(x_LL, x_UL, n)

V_p = np.sqrt((8 * k * x_0**3) / (27 * eps_0 * eps_r * A_0))  # Pulldown voltag
V_2 = V_p * 2/3  # Some voltage between V = 0 and V = V_p
x_p = x_0 * 2/3  # x where two curves intersect at V = V_p

# Calculate lambda = x/x_0 (stretch ratio in thickness direction)
lambda_vals = x / x_0

# Calculate forces - use ABSOLUTE VALUES as in the MATLAB plot
# Linear mechanical force (Hookean) - absolute value
F_m_linear = np.abs(k * (x_0 - x))

# Nonlinear mechanical force: Fm = (A * E / 3) * (lambda - 1/lambda^2)
# Take absolute value and ensure positive force
F_m_nonlinear = np.abs((A_0 * E / 3) * (lambda_vals - 1/lambda_vals**2))

# REVISED Electrical forces: Fe = (eps_0 * eps_r * A_0 * V^2) / (2 * x_0^2 * la
F_e_V_p = np.abs((eps_0 * eps_r * A_0 * V_p**2) / (2 * x_0**2 * lambda_vals**3)
F_e_V_2 = np.abs((eps_0 * eps_r * A_0 * V_2**2) / (2 * x_0**2 * lambda_vals**3)

# Create plot
plt.figure(figsize=(12, 8))
plt.plot(x, F_m_linear, 'b--', label='F_m (linear)', linewidth=2)
plt.plot(x, F_m_nonlinear, 'r-', label='F_m (nonlinear)', linewidth=2)
plt.plot(x, F_e_V_p, 'g-', label='F_e (V = V_p)', linewidth=2)
plt.plot(x, F_e_V_2, 'm-', label='F_e (V = V_p*2/3)', linewidth=2)
plt.axvline(x=x_p, color='k', linestyle='--', label=f'x_p = {x_p:.1e} m')

plt.ylim([0, 100])
plt.xlim([x_LL, x_UL])
plt.xlabel('x (m)')
plt.ylabel('Absolute value of Force (N)')
plt.legend()
plt.title('Force vs. Displacement (with revised F_e)')
plt.grid(True, alpha=0.3)

# Add x10^-4 notation to x-axis as in your plot
from matplotlib.ticker import FuncFormatter
plt.gca().xaxis.set_major_formatter(FuncFormatter(lambda x, p: f'{x*1e4:.1f}'))
plt.gca().set_xlabel('x (m) $times 10^{-4}$')
```

```
plt.show()

# Print values for comparison and debugging
print(f"Pulldown voltage V_p = {V_p:.2f} V")
print(f"Stiffness k = {k:.2f} N/m")
print(f"Intersection point x_p = {x_p:.2e} m")
print(f"Original area A_0 = {A_0:.2e} m²")
print(f"Lambda range: {lambda_vals[0]:.3f} to {lambda_vals[-1]:.3f}")
print(f"F_m_linear range: {F_m_linear[0]:.2f} to {F_m_linear[-1]:.2f} N")
print(f"F_m_nonlinear range: {F_m_nonlinear[0]:.2f} to {F_m_nonlinear[-1]:.2f} I
print(f"F_e_V_p at x_min: {F_e_V_p[0]:.2f} N, at x_max: {F_e_V_p[-1]:.2f} N")
```



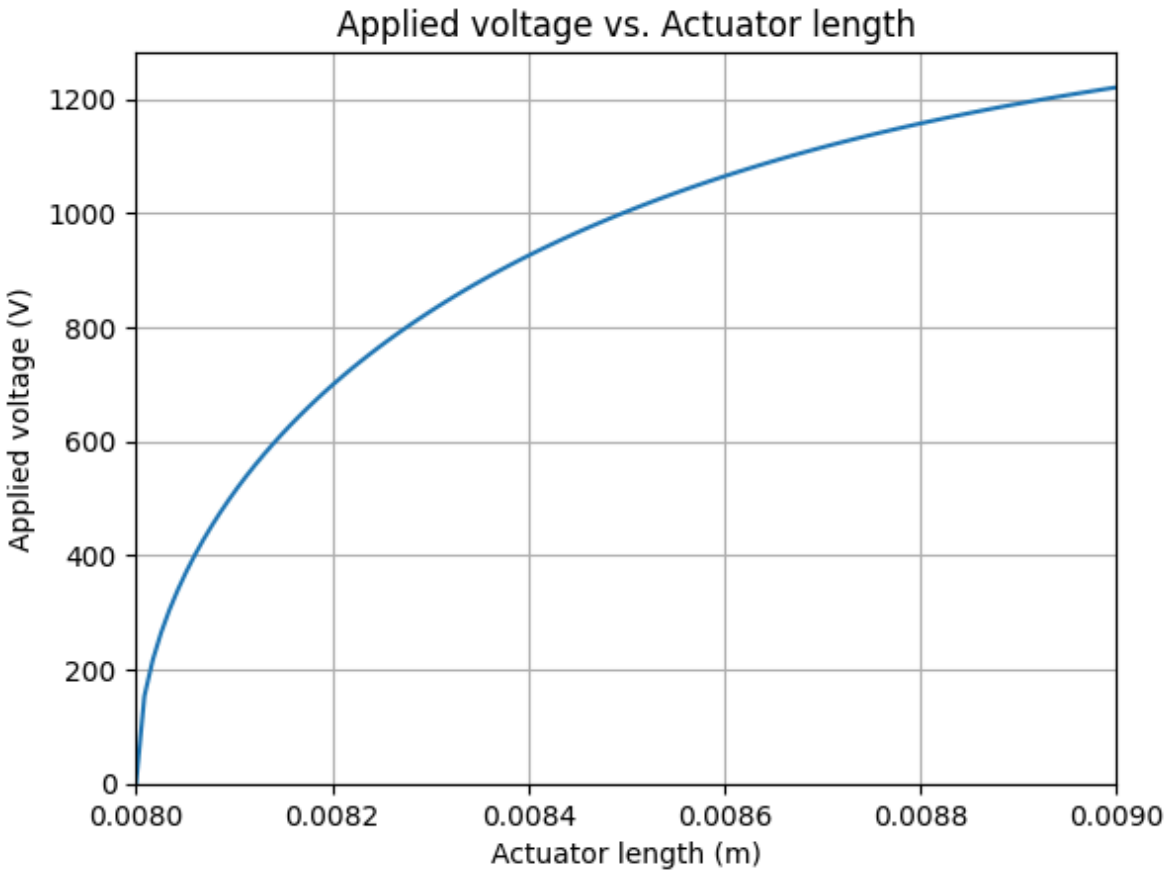

```
Pulldown voltage V_p = 8999.14 V
Stiffness k = 254545.45 N/m
Intersection point x_p = 1.47e-04 m
Original area A_0 = 4.00e-04 m²
Lambda range: 0.182 to 1.000
F_m_linear range: 45.82 to 0.00 N
F_m_nonlinear range: 561.27 to 0.00 N
F_e_V_p at x_min: 1380.30 N, at x_max: 8.30 N
```

Problem 3 - Part B

In [6]:

```
import numpy as np
import matplotlib.pyplot as plt

# Problem 3 b )
E = 140e3  # Young's Modulus, Pa
t_0 = 220e-6  # Initial thickness of the actuator, m
eps_0 = 8.854e-12  # Permissivity of free space, Farad/m
eps_r = 2.8  # Relative permissivity, dimensionless
```

```
L_0 = 8e-3  # Initial length of actuator, m
R_0 = 3e-3  # Initial radius of actuator, m
n = 7  # Number of capacitor layers

x_0 = t_0 / 7  # Initial thickness of each capacitor layer

numpoints = 100  # Number of points for plotting
L_max = 9e-3  # Max L for plotting
lambda_min = (L_0 / L_max) ** 2  # min lambda for plotting; note that larger L =

lambda_vals = np.linspace(lambda_min, 1, numpoints)

U = np.zeros(numpoints)
L_vals = np.zeros(numpoints)

for i in range(numpoints):
    U[i] = np.sqrt((2 * E * x_0 ** 2 * lambda_vals[i] ** 3) / (3 * eps_0 * eps_
    L_vals[i] = L_0 / np.sqrt(lambda_vals[i])

plt.figure()
plt.plot(L_vals, U)
plt.xlabel('Actuator length (m)')
plt.ylabel('Applied voltage (V)')
plt.title('Applied voltage vs. Actuator length')
plt.grid(True)
plt.xlim(8e-3, 9e-3)
plt.ylim(bottom=0)

plt.show()
```



Problem 3 - Part C

In [12]:
```
import numpy as np

# --- Given parameters ---
E = 140e3           # Pa
t0 = 220e-6         # m
eps0 = 8.854e-12    # F/m
epsr = 2.8
L0 = 8e-3           # m
n = 7
x0 = t0 / n         # Initial thickness of each capacitor layer

# --- Desired elongation ---
delta_L = 1e-3      # 1 mm extension
L = L0 + delta_L    # Final length

# --- Calculate lambda ---
lambda_val = (L0 / L)**2  # Note: lambda = (L0/L)^2 for extension

# --- Voltage Formula ---
def V_required(lambda_val):
    term = (2 * E * x0**2 * lambda_val**3) / (3 * eps0 * epsr) * abs((lambda_va
    if term < 0:
        raise ValueError("Negative value under square root")
    return np.sqrt(term)

# --- Compute voltage ---
V = V_required(lambda_val)

# --- Display result ---
print(f"Initial length L0 = {L0*1e3:.1f} mm")
print(f"Final length L = {L*1e3:.1f} mm")
print(f"Elongation ΔL = {delta_L*1e3:.1f} mm")
print(f"Lambda = {lambda_val:.4f}")
print(f"Required applied voltage = {V:.2f} V ({V/1000:.2f} kV)")
```

```
Initial length L0 = 8.0 mm
Final length L = 9.0 mm
Elongation ΔL = 1.0 mm
Lambda = 0.7901
Required applied voltage = 1220.20 V (1.22 kV)
```

Problem 3 - Part D

In [11]:
```
import sympy as sp

# Problem 3 d )
E = 140e3  # Young's Modulus, Pa
t_0 = 220e-6  # Initial thickness of the actuator, m
eps_0 = 8.854e-12  # Permissivity of free space, Farad/m
eps_r = 2.8  # Relative permissivity, dimensionless
```

```
# Relative permittivity, dimensionless
L_0 = 8e-3  # Initial length of actuator, m
R_0 = 3e-3  # Initial radius of actuator, m
n = 7  # Number of capacitor layers

x_0 = t_0 / 7  # Initial thickness of each capacitor layer

# Define symbolic variables
lambda_sym, U_sym = sp.symbols('lambda U')

# Define equations
force_eq = sp.Eq(E/3*(1/lambda_sym**2 - lambda_sym), (eps_0*eps_r*U_sym**2)/(2*
force_deriv_eq = sp.Eq(E/3*(-2/lambda_sym**3 - 1), -(3*eps_0*eps_r*U_sym**2)/(2

# Solve the system of equations
solutions = sp.solve([force_eq, force_deriv_eq], [lambda_sym, U_sym])

# Extract solutions
sol_lambda, sol_U = solutions[0]  # Get the first solution

print(f"Lambda solution: {sol_lambda}")
print(f"U solution: {abs(sol_U)} V")
```

```
Lambda solution: 0.629960524947437
U solution: 1325.50556201050 V
```

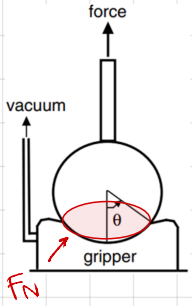
Problem 1 Part A

Assumptions

* uniform contraction when vacuum is applied
* pinching occurs in a thin band of width d near the contact line
* pinched region acts like an o-ring pressing against the sphere
* the stress σ^* is uniform in the pinching region
* the sphere is rigid compare to the gripper material
* friction follows Coulomb's law w/ coefficient μ

Variables

* R = sphere radius
* θ = contact angle (angle from vertical to contact line
* μ = coefficient of friction
* σ^* = pinching stress in the O-ring region
* d = width of pinching region
* A_0 = area of the pinching region
* A^* = horizontal cross-sec area inside contact line



① Porous Sphere (Friction only)

A pinching region forms an O-ring around the sphere at angle θ

$$\Rightarrow A_0 = 2\pi R d \sin\theta$$

Normal force is the stress times area times the component normal to the surface

$$\Rightarrow F_N = \sigma^* A_0 \sin\theta = \sigma^*(2\pi R d \sin\theta)\sin\theta = 2\pi R d \sigma^* \sin^2\theta$$

$F_f = \mu F_N$, only the vertical component contributes to holding

$$\therefore F_f = F_N(\mu\sin\theta - \cos\theta) = 2\pi R d \sigma^* \sin^2\theta(\mu\sin\theta - \cos\theta)$$

Note: gripping only occurs when $\mu\sin\theta - \cos\theta > 0 \Rightarrow \theta > \theta_c = \arctan\left(\frac{1}{\mu}\right)$

$$\therefore \boxed{F_h = 2\pi R d \sigma^*(\mu\sin\theta - \cos\theta)\sin^2\theta}$$

② Solid sphere (Friction + Suction)

For a sphere, the horizontal cross-sectional area inside the contact line is:

$$A^* = \pi R^2 \sin^2\theta$$

The suction force is $F_s = P_g A^*$, where P_g is the gap pressure. Relating P_g to pinching stress

$$P_g = \frac{F_f}{A_0} = \sigma^* \sin\theta(\mu\sin\theta - \cos\theta)$$

$$\Rightarrow F_s = P_g A^* = \pi R^2 \sigma^*(\mu\sin\theta - \cos\theta)\sin^3\theta$$

$$\therefore F_h = F_s + F_f = \pi R^2 \sigma^*(\mu\sin\theta - \cos\theta)\sin^3\theta + 2\pi R d \sigma^*(\mu\sin\theta - \cos\theta)\sin^2\theta$$

$$\boxed{F_h = \pi R^2 \sigma^*(\mu\sin\theta - \cos\theta)\sin^3\theta\left(1 + \frac{2d}{R\sin\theta}\right)}$$

Unclear details from research paper:

→ It does not explain how pinching width d is determined

→ How σ^* relates to P_{jam} is not quantitatively derived

→ The transition between bending-dominated and stretching-dominated interlocking for $\theta > \pi/2$ lacks criteria

Part B Look at code!! Plot holding force vs contact angle

Part C // Plot critical angle vs friction coefficient

As μ increases, θ_c decreases significantly. A lower θ_c means the gripper can hold objects with shallower contact angles Gripping begins at smaller deformation angles. It can also grip flatter objects and less precise positioning required

Part D θ_c decreases 6.8°, holding force increase by 25-30% across all contact angles. Therefore, gripper can engage earlier with shallower contact, and more robust gripping of objects with low curvature

Part E Alternative Gripper Design

Adding micro-scale patterns or textures to the gripper membrane surface to enhance geometric interlocking at smaller angles

Expected Effects on F_h (holding force)

→ texture increases effective μ through mechanical interlocking

→ better conformal contact for vacuum sealing

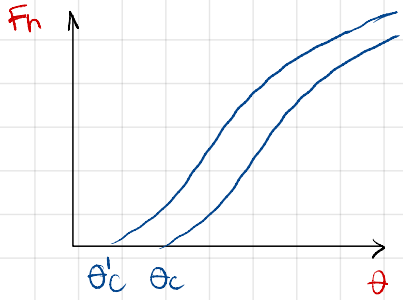
→ small-scale geometric constraints

Modified Model $\mu_{eff} = \mu_{adhesive} + \mu_{texture}$

Mtexture depends on:
→ texture geometry (spacing, shape, height)
→ material compliance
→ Local curvature of object surface

Modified Equation ⇒ $F_n = \pi R^2 \sigma^* (M_{eff} \sin\theta - \cos\theta) \sin^3\theta \left(1 + \frac{2d_{eff}}{R\sin\theta}\right)$

where deff may increase due to better conformal contact



We can get:
* Lower critical angle
* Higher maximum holding force

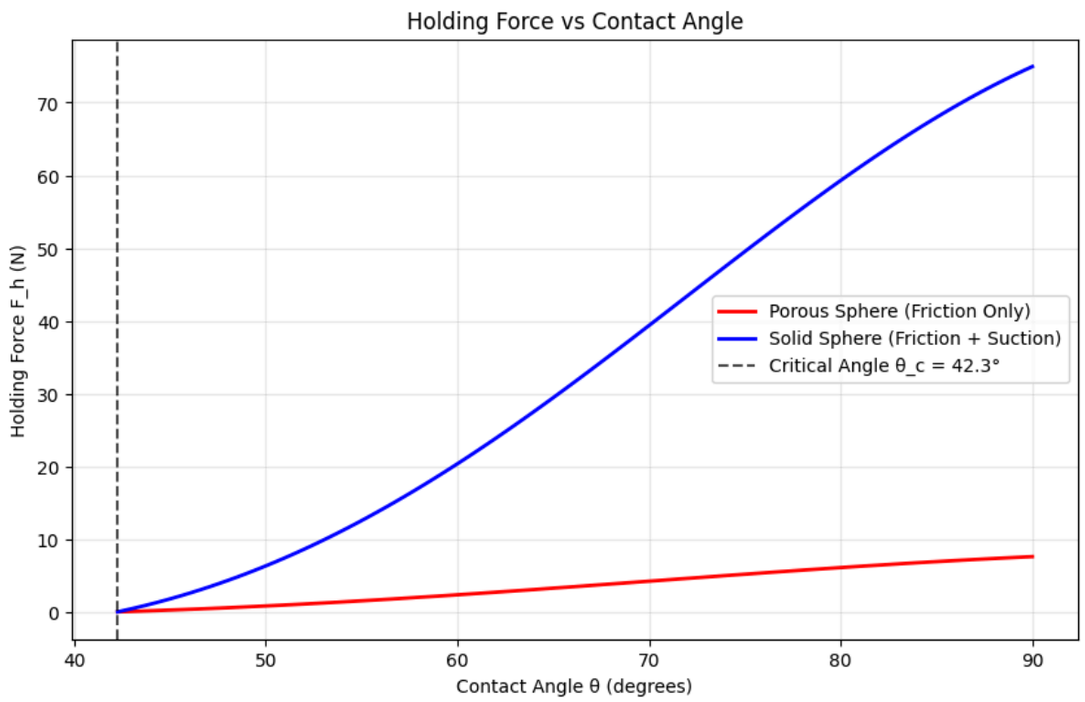
Check code for plots !!

Reasoning: Micro-scale features can engage with object surface roughness. Texture provides additional gripping mechanisms before full macroscopic contact is established. Lastly, the modification would be most beneficial for smooth, hard to grip surfaces

Open in Colab

Problem 1 - Part B

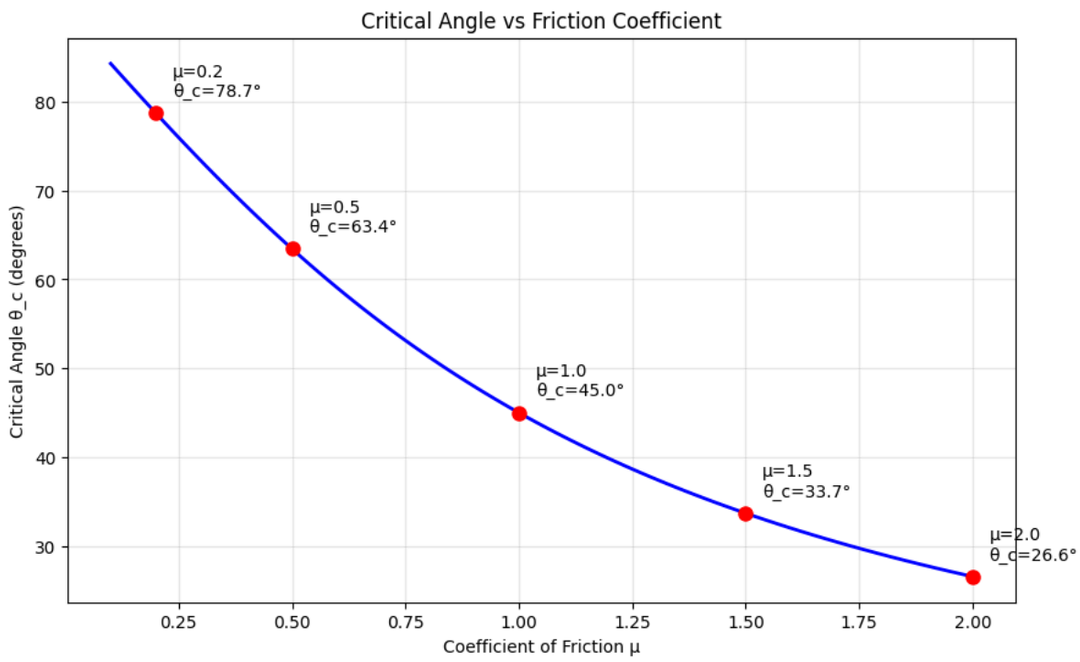
In [11]:
```
import numpy as np
import matplotlib.pyplot as plt

# Given parameters
mu = 1.1
sigma_star = 54e3  # Pa
d = 1.07e-3  # m
R = 19e-3  # m

# Critical angle
theta_c = np.arctan(1/mu)

# Create theta range from theta_c to pi/2
theta = np.linspace(theta_c, np.pi/2, 100)

# Porous sphere (friction only)
F_h_porous = 2 * np.pi * R * d * sigma_star * (mu * np.sin(theta) - np.cos(theta

# Solid sphere (friction + suction)
F_h_solid = (np.pi * R**2 * sigma_star * (mu * np.sin(theta) - np.cos(theta)) *
             (1 + (2*d)/(R * np.sin(theta))))

# Convert to Newtons and plot
plt.figure(figsize=(10, 6))
plt.plot(np.degrees(theta), F_h_porous, 'r-', linewidth=2, label='Porous Sphere
plt.plot(np.degrees(theta), F_h_solid, 'b-', linewidth=2, label='Solid Sphere (
plt.axvline(np.degrees(theta_c), color='k', linestyle='--', alpha=0.7, label=f'

plt.xlabel('Contact Angle θ (degrees)')
plt.ylabel('Holding Force F_h (N)')
plt.title('Holding Force vs Contact Angle')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print(f"Critical angle θ_c = {np.degrees(theta_c):.2f}°")
print(f"Maximum holding force (porous): {np.max(F_h_porous):.2f} N")
print(f"Maximum holding force (solid): {np.max(F_h_solid):.2f} N")
```




```
Critical angle θ_c = 42.27°
Maximum holding force (porous): 7.59 N
Maximum holding force (solid): 74.95 N
```

Problem 1 - Part C

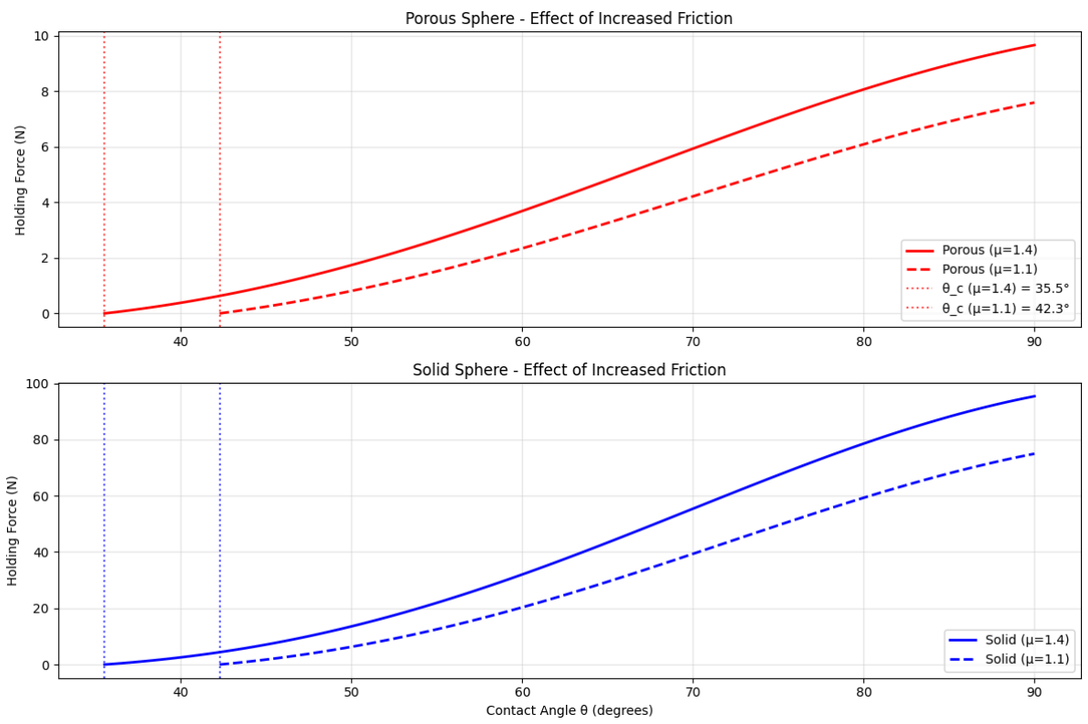
In [12]:
```
import numpy as np
import matplotlib.pyplot as plt

# Physically achievable range of friction coefficients
# Rubber on various surfaces can achieve μ = 0.1 to 2.0+
mu_range = np.linspace(0.1, 2.0, 100)
theta_c_range = np.arctan(1/mu_range)

plt.figure(figsize=(10, 6))
plt.plot(mu_range, np.degrees(theta_c_range), 'b-', linewidth=2)
plt.xlabel('Coefficient of Friction μ')
plt.ylabel('Critical Angle θ_c (degrees)')
plt.title('Critical Angle vs Friction Coefficient')
plt.grid(True, alpha=0.3)

# Mark some realistic values
realistic_mu = [0.2, 0.5, 1.0, 1.5, 2.0]
for mu_val in realistic_mu:
    theta_val = np.degrees(np.arctan(1/mu_val))
    plt.plot(mu_val, theta_val, 'ro', markersize=8)
    plt.annotate(f'μ={mu_val}\nθ_c={theta_val:.1f}°',
                 (mu_val, theta_val),
                 xytext=(10, 10), textcoords='offset points')

plt.show()
```



Problem 1 - Part D

In [3]:
```
# Parameters with increased friction
mu_high = 1.4
sigma_star = 54e3  # Pa
d = 1.07e-3  # m
R = 19e-3  # m

# New critical angle
theta_c_high = np.arctan(1/mu_high)

# Theta range from new critical angle to pi/2
theta_high = np.linspace(theta_c_high, np.pi/2, 100)

# Calculate holding forces
F_h_porous_high = 2 * np.pi * R * d * sigma_star * (mu_high * np.sin(theta_high)
F_h_solid_high = (np.pi * R**2 * sigma_star * (mu_high * np.sin(theta_high) - np
                  (1 + (2*d)/(R * np.sin(theta_high))))

# Compare with original μ = 1.1
```

```
mu_orig = 1.1
theta_c_orig = np.arctan(1/mu_orig)
theta_orig = np.linspace(theta_c_orig, np.pi/2, 100)

F_h_porous_orig = 2 * np.pi * R * d * sigma_star * (mu_orig * np.sin(theta_orig)
F_h_solid_orig = (np.pi * R**2 * sigma_star * (mu_orig * np.sin(theta_orig) - np
                  (1 + (2*d)/(R * np.sin(theta_orig))))

# Plot comparison
plt.figure(figsize=(12, 8))

plt.subplot(2, 1, 1)
plt.plot(np.degrees(theta_high), F_h_porous_high, 'r-', linewidth=2, label='Porc
plt.plot(np.degrees(theta_orig), F_h_porous_orig, 'r--', linewidth=2, label='Por
plt.axvline(np.degrees(theta_c_high), color='r', linestyle=':', alpha=0.7, label
plt.axvline(np.degrees(theta_c_orig), color='r', linestyle=':', alpha=0.7, label
plt.ylabel('Holding Force (N)')
plt.title('Porous Sphere - Effect of Increased Friction')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(2, 1, 2)
plt.plot(np.degrees(theta_high), F_h_solid_high, 'b-', linewidth=2, label='Solid
plt.plot(np.degrees(theta_orig), F_h_solid_orig, 'b--', linewidth=2, label='Sol
plt.axvline(np.degrees(theta_c_high), color='b', linestyle=':', alpha=0.7)
plt.axvline(np.degrees(theta_c_orig), color='b', linestyle=':', alpha=0.7)
plt.xlabel('Contact Angle θ (degrees)')
plt.ylabel('Holding Force (N)')
plt.title('Solid Sphere - Effect of Increased Friction')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Critical angle reduction: {np.degrees(theta_c_orig - theta_c_high):.1f}°
print(f"Maximum force increase (porous): {((np.max(F_h_porous_high)/np.max(F_h_p
print(f"Maximum force increase (solid): {((np.max(F_h_solid_high)/np.max(F_h_sol
```



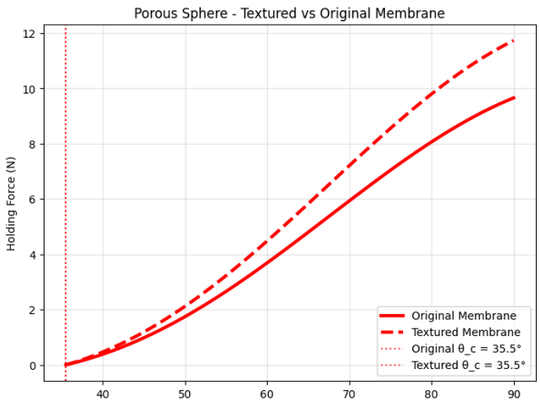

```
Critical angle reduction: 6.7°
Maximum force increase (porous): 27.3%
Maximum force increase (solid): 27.3%
```

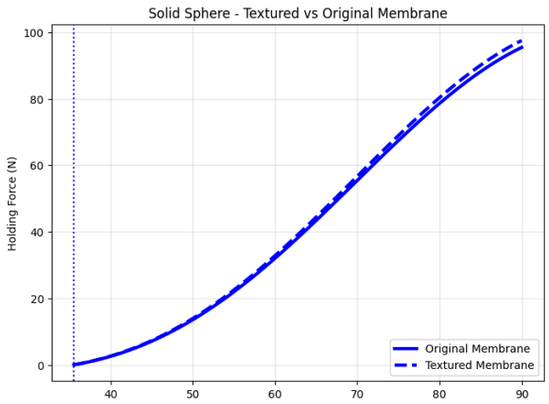
In [2]:

```
import numpy as np
import matplotlib.pyplot as plt

# Base parameters from the paper
sigma_star = 54e3  # Pa
```

```
d = 1.07e-3  # m
R = 19e-3  # m
mu_original = 1.4

# Textured membrane parameters
mu_textured = 1.4  # Increased effective friction
d_effective = 1.3e-3  # Increased effective contact width due to better sealing

def holding_force_porous(theta, mu, d_eff):
    """Holding force for porous sphere (friction only)"""
    return 2 * np.pi * R * d_eff * sigma_star * (mu * np.sin(theta) - np.cos(the

def holding_force_solid(theta, mu, d_eff):
    """Holding force for solid sphere (friction + suction)"""
    return (np.pi * R**2 * sigma_star * (mu * np.sin(theta) - np.cos(theta)) * r
            (1 + (2*d_eff)/(R * np.sin(theta))))

# Calculate critical angles
theta_c_original = np.arctan(1/mu_original)
theta_c_textured = np.arctan(1/mu_textured)

# Create theta ranges
theta_original = np.linspace(theta_c_original, np.pi/2, 100)
theta_textured = np.linspace(theta_c_textured, np.pi/2, 100)

# Calculate holding forces
F_porous_orig = holding_force_porous(theta_original, mu_original, d)
F_solid_orig = holding_force_solid(theta_original, mu_original, d)

F_porous_text = holding_force_porous(theta_textured, mu_textured, d_effective)
F_solid_text = holding_force_solid(theta_textured, mu_textured, d_effective)

# Plot comparison
plt.figure(figsize=(14, 10))

# Porous sphere comparison
plt.subplot(2, 2, 1)
plt.plot(np.degrees(theta_original), F_porous_orig, 'r-', linewidth=3, label='Or
plt.plot(np.degrees(theta_textured), F_porous_text, 'r--', linewidth=3, label='
plt.axvline(np.degrees(theta_c_original), color='r', linestyle=':', alpha=0.7,
           label=f'Original θ_c = {np.degrees(theta_c_original):.1f}°')
plt.axvline(np.degrees(theta_c_textured), color='r', linestyle=':', alpha=0.7,
           label=f'Textured θ_c = {np.degrees(theta_c_textured):.1f}°')
plt.ylabel('Holding Force (N)')
plt.title('Porous Sphere - Textured vs Original Membrane')
plt.legend()
plt.grid(True, alpha=0.3)

# Solid sphere comparison
plt.subplot(2, 2, 2)
plt.plot(np.degrees(theta_original), F_solid_orig, 'b-', linewidth=3, label='Ori
plt.plot(np.degrees(theta_textured), F_solid_text, 'b--', linewidth=3, label='Te
plt.axvline(np.degrees(theta_c_original), color='b', linestyle=':', alpha=0.7)
plt.axvline(np.degrees(theta_c_textured), color='b', linestyle=':', alpha=0.7)
plt.ylabel('Holding Force (N)')
plt.title('Solid Sphere - Textured vs Original Membrane')
plt.legend()
plt.grid(True, alpha=0.3)

# Force enhancement percentage
theta_common = np.linspace(max(theta_c_original, theta_c_textured), np.pi/2, 10
F_porous_orig_common = holding_force_porous(theta_common, mu_original, d)
F_porous_text_common = holding_force_porous(theta_common, mu_textured, d_effecti
F_solid_orig_common = holding_force_solid(theta_common, mu_original, d)
F_solid_text_common = holding_force_solid(theta_common, mu_textured, d_effective


plt.tight_layout()
plt.show()

# Quantitative analysis
print("=== TEXTURED MEMBRANE EFFECTS ===")
print(f"Critical angle reduction: {np.degrees(theta_c_original - theta_c_texture
print(f"Effective contact width increase: {(d_effective/d - 1)*100:.0f}%")
```

```
max_theta = np.pi/2
F_porous_max_orig = holding_force_porous(max_theta, mu_original, d)
F_porous_max_text = holding_force_porous(max_theta, mu_textured, d_effective)
F_solid_max_orig = holding_force_solid(max_theta, mu_original, d)
F_solid_max_text = holding_force_solid(max_theta, mu_textured, d_effective)

print(f"\nMaximum holding force at θ = 90°:")
print(f"Porous sphere - Original: {F_porous_max_orig:.2f} N")
print(f"Porous sphere - Textured: {F_porous_max_text:.2f} N")

print(f"\nSolid sphere - Original: {F_solid_max_orig:.2f} N")
print(f"Solid sphere - Textured: {F_solid_max_text:.2f} N")
```






```
=== TEXTURED MEMBRANE EFFECTS ===
Critical angle reduction: 0.0°
Effective contact width increase: 21%

Maximum holding force at θ = 90°:
Porous sphere - Original: 9.66 N
Porous sphere - Textured: 11.73 N

Solid sphere - Original: 95.40 N
Solid sphere - Textured: 97.47 N
```